NEAT: NeuroEvolution of Neural Network Topologies

Contents

1 Project Overview **3**

2 Terminology **3**
- 2.1 Overview of NEAT . . . 5
- 2.2 How NEAT Evolves Networks . . . 5

3 Project Structure **6**
- 3.1 NEAT Package . . . 7
- 3.2 Models Package . . . 7
- 3.3 Simulations Package . . . 8
- 3.4 Visualization Package . . . 8
- 3.5 Experiments Package . . . 8

4 Genome Representation **8**

5 Model Execution **9**
- 5.1 Feedforward Cell . . . 9
- 5.2 Recurrent Cell . . . 9
- 5.3 Long Short-Term Memory (LSTM) Cell . . . 10
- 5.4 Batched Execution . . . 10
- 5.5 Model State and Episode Reset . . . 10

6 Simulation and Episode Evaluation **11**

7 Trajectory Recording and Replay **12**

8 Topology Visualization **12**

9 API Reference **12**
- 9.1 NEAT Package . . . 12
  - 9.1.1 `neat/genome.py` . . . 12
  - 9.1.2 `neat/genes.py` . . . 14
  - 9.1.3 `neat/innovation.py` . . . 15
  - 9.1.4 `neat/species.py` . . . 15
  - 9.1.5 `neat/neat.py` . . . 16
- 9.2 Models Package . . . 17
  - 9.2.1 `models/model.py` . . . 17
  - 9.2.2 `models/compiler.py` . . . 18
  - 9.2.3 `models/cells.py` . . . 18
- 9.3 Simulations Package . . . 18

9.3.1 `simulations/simulation.py` 18
9.4 Visualization Package 20
9.4.1 `visualization/topology.py` 20
9.4.2 `visualization/replay.py` 20

References **21**

1 Project Overview

This project implements a custom version of NeuroEvolution of Augmenting Topologies (NEAT).

The implementation will support networks containing feedforward (FFNN), recurrent (RNN), and long short-term memory (LSTM) cells. PyTorch will be used for tensor operations and GPU acceleration, but its gradients and backpropagation will not be necessary for the genetic algorithm.

This project is designed to evolve a neural network (NN)for an arbitrary user defined simulation. The main test cases are pole balancing and maze solving.

2 Terminology

Here we define the common terms used for this project.

Tensor
A tensor is a generalize array for numerical data, extending vectors and matrices to higher dimensions. This project uses PyTorch tensors to store and manipulate network weights and values over batches of agents. Tensor operations can be executed on a GPU, allowing many numerical operations to be performed in parallel.

Network topology
The network topology is the directed graph describing how information flows through a neural network. It consists of nodes and directed edges: nodes perform computations, while edges specify which nodes send information to which other nodes. The topology is distinct from the numerical weights stored on the edges. Two networks can have the same topology but different weights.

Node / Cell
A node, also called a cell in this project, is one computational unit in the network. It receives values from connected nodes, performs a computation, and produces an output. A cell can use different computational rules: an FFNN cell performs a feedforward operation, an RNN cell receives a hidden state between timesteps, and an LSTM cell maintains both a hidden state and a cell state.

NEAT
NeuroEvolution of Augmenting Topologies (NEAT) is an evolutionary algorithm that creates neural networks by evolving both their connection weights and their structures. NEAT maintains a population of candidate networks and improves them over generations using a survival of the fittest approach. Networks are evaluated in a task, better-performing networks are more likely to contribute to the next generation, and mutations can change weights or add and disable structural elements. This allows the algorithm to discover both network parameters and network topologies.

Genome
A genome is the description of one candidate neural network in NEAT. It contains the information describing its topology such as its nodes and connections, but also includes connection weights, enabled connections, and innovation numbers. The genome is the object that NEAT mutates, compares, and combines during evolution rather than operating directly on the compiled model itself.

Gene
A gene is one component of a genome that describes a particular part of the network. The gene contains the evolutionary information used by NEAT in addition to the model information. A genome is the collection of these genes that together describes an entire network. This implementation uses 2 genes: a node genes to describe individual nodes and connection genes to describe directed connections between nodes.

Innovation number
An innovation number is an identifier assigned to a structural change during evolution. When a mutation adds a connection, NEAT assigns that connection an innovation number. If the same structural mutation occurs independently in another genome, NEAT should recognize that to align the corresponding genes when comparing genomes or combining them during reproduction.

Species
A species is a group of genomes with sufficiently similar network structures. NEAT divides the population into species using a compatibility distance that measures structural and parameter differences between genomes. This grouping helps preserve diversity in different structural approaches: a newly developed network topology can compete primarily with similar networks rather than immediately being eliminated by a larger group of already well-performing networks.

Population
The population is the collection of genomes being evolved in one NEAT run. Each generation, the population's genomes are evaluated, assigned to species, and used to produce the next generation through selection, crossover, and mutation.

Fitness
Fitness is a numerical measure of how well a genome performs in the simulation task. The simulation calculates fitness from an agent's behavior.

Batch
In this project a batch does not refer to the training data samples being evaluated in parallel. Rather a batch is the collection of agents whose networks are evaluated together during a simulation run. Each agent is controlled by its own model, and the models may have different genomes and network topologies making gpu parallelization trickier.

Agent
An agent is an individual entity acting within a simulation. It receives observations from the simulation, passes them to its assigned model, and uses the model's output as its action.

State
The set of information used to define the simulation at any given time. A simulation-state representation is not unique. The simulation-state can contain separate and additional information from the information fed to the model as observations.

Model
A model is an executable instance of a network described by a genome. Where as the genome is the evolutionary representation, while the model is the computational representation.

Episode
An episode is one complete evaluation of an agent in a simulation, beginning with a reset or initial state and ending when the agent terminates or reaches the simulation's time limit. A genome can be evaluated over multiple episodes using different initial conditions before its fitness is aggregated for evolution.

Trajectory
A trajectory is the list of an agent's simulation-state over an episode. It can be represented as a tensor with a state vector for multiple simulation timesteps, with an additional agent dimension when multiple agents are recorded together.

Recurrent state
Recurrent state is information retained by a model between timesteps. An FFNN has no recurrent state, an RNN retains a hidden state, and an LSTM retains both a hidden state and a cell state. This state allows a model to use information from earlier observations when producing its current action. Recurrent state is reset at the beginning of each episode.

Crossover
Crossover is the process of combining genes from two parent genomes to produce an offspring genome. NEAT uses innovation numbers to identify corresponding connection genes in the parents, allowing their network descriptions to be aligned even when their topologies differ.

Mutation
Mutation modifies a genome to create a new candidate network. Weight mutation changes numerical connection parameters, while structural mutations can add nodes or connections and disable existing connections.

2.1 Overview of NEAT

An example of a set of mutations to a given genome:

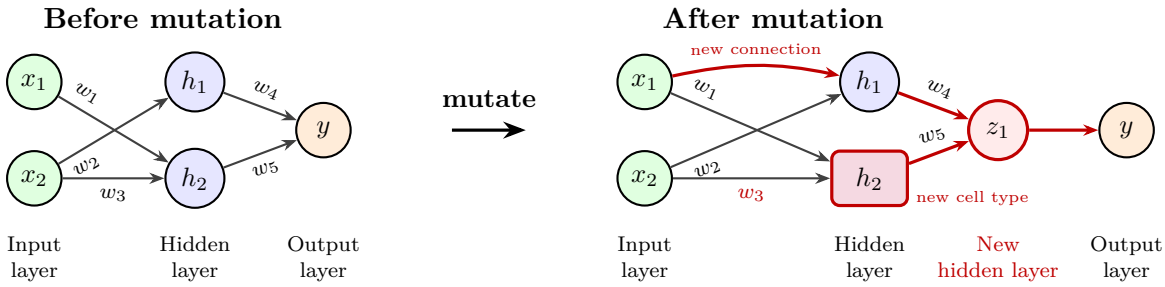


Figure 1: A genome before and after three mutations: adding a connection, changing an FFNN cell to an LSTM cell, and adding a new hidden layer containing one node.

In this example each node and its connections are referred to as genes and their collection making up the topology and evolutionary state information is the genome.

This example shows the type of mutations possible: change weights, add or remove connections, add or remove nodes, add or remove layers, change cell type.

The mutation shown adds a new connection from x_1 to h_2. The resulting genome has a different topology and therefore describes a different network. The original genes are retained, while the new connection is represented by an additional connection gene with its own innovation number.

A wight update is a mutation a genome without changing its topology.

2.2 How NEAT Evolves Networks

NEAT evolves a population of genomes over multiple generations. Each generation evaluates the current population, uses fitness to guide reproduction, and produces a new population through crossover and mutation.

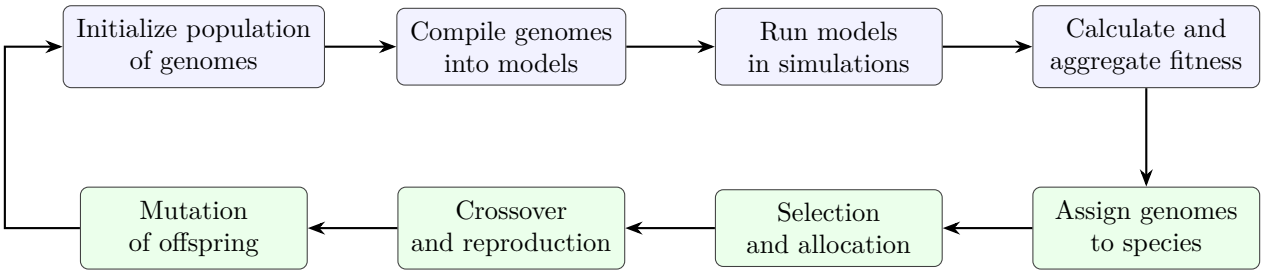


Figure 2: Simplified NEAT evolutionary workflow. The top row shows evaluation and fitness calculation. The bottom row shows how the population is organized and how the next generation is produced.

The workflow steps are:

1. **Initialize:** Create an initial population of genomes describing candidate neural networks.
2. **Compile:** Convert each genome into an executable model.
3. **Evaluate:** Run the models in the selected simulation. Each model controls an agent through one or more episodes.
4. **Calculate fitness:** Use the simulation results to assign fitness to each genome. If a genome is evaluated over multiple episodes, its episode fitness values are aggregated.
5. **Speciate:** Group structurally similar genomes into species using compatibility distance.
6. **Select:** Allocate reproduction opportunities based on fitness, species performance, and the configured selection strategy.
7. **Reproduce:** Create offspring by selecting parent genomes and combining their genes through crossover.
8. **Mutate:** Modify offspring genomes through weight mutations and structural mutations.
9. **Repeat:** Evaluate the resulting population as the next generation.

This process repeats until a configured stopping condition is reached, such as a maximum number of generations or a target fitness.

3 Project Structure

The project is organized into the following structure.

```
project/
    neat/
        neat.py
        genome.py
        genes.py
        innovation.py
        species.py

    models/
        model.py
```

```
        compiler.py
        cells.py

    simulations/
        simulation.py
        maze.py
        pole_balancing.py

    visualization/
        topology.py
        replay.py

    experiments/
        train.py
        evaluate.py
        configs/
```

3.1 NEAT Package

`NEAT` – `neat.py`
Responsible for managing the population and evolution algorithm defining `initialize_population()`, `evaluate_population()`, `speciate()`, `reproduce()`, and `evolve()`.

`Genome` – `genome.py`
Stores the node and connection genes describing one candidate network. Implements structural and parameter mutation, crossover, compatibility distance, validation, and serialization.

`NodeGene`, `ConnectionGene` – `genes.py`
Simple dataclass objects for storing individual nodes and connections information.

`InnovationTracker` – `innovation.py`
Assigns and tracks historical innovation numbers so structural mutations can be identified consistently across genomes.

`Species` – `species.py`
Maintains a group of structurally similar genomes, including its representative, membership, adjusted fitness, and stagnation information.

3.2 Models Package

`Model` – `model.py`
Represents an executable network compiled from a genome. Exposes the forward-pass interface and manages runtime recurrent state, including state reset between episodes.

`Compiler` – `compiler.py`
Validates a genome and converts its graph representation into an execution plan. Determines a valid topological order, organizes connections, and prepares the tensor data required by execution.

`Cells` – `cells.py`
Implements the tensor operations used to evaluate FFNN, RNN, and LSTM cells. Coordinates execution according to the compiled plan and supports batched evaluation where network structures are compatible.

3.3 Simulations Package

`Simulation` – `simulation.py`
Defines the common simulation interface and generic episode loop. Handles model execution, observations, actions, agent tracking, fitness accumulation, and optional recording. This is an abstract class that must be inherited with concrete implementations.

`MazeSimulation` – `maze.py`
Implements the environment-specific behavior for maze navigation.

`PoleBalancingSimulation` – `pole_balancing.py`
Implements the environment-specific behavior for pole balancing.

3.4 Visualization Package

`TopologyVisualization` – `topology.py`
Plots a genome as a directed graph, showing nodes, connections, cell types, and optionally connection weights, innovation numbers, and disabled connections.

`Replay` – `replay.py`
Loads a saved episode record and replays the agent's actions in the corresponding simulation. Replay uses the recorded actions rather than requiring the neural network to be executed again.

3.5 Experiments Package

The `experiments/` package contains entry points for training and evaluating networks.

4 Genome Representation

A genome stores a network as a directed graph consisting of node genes and connection genes. Each node has a unique identifier and a role. Input and output nodes define the network interface and thus have more limited mutations allow on them, while hidden nodes perform computations and can experience any mutation.

Each node gene stores information such as:

- A unique node identifier.
- Its role: input, hidden, or output.
- Its computational type: FFNN, RNN, or LSTM where applicable.
- Its activation function, where applicable.
- Any node-specific parameters, such as biases.

Each connection gene stores information such as:

- A unique connection identifier.
- Source and target node identifiers.
- Its weight or gate-specific weights.
- Whether the connection is enabled.
- Its innovation number.

The genome is the evolutionary representation. This separation allows genomes to be mutated, compared, serialized, and inspected independently of model execution.

5 Model Execution

Each node represents a cell, and each directed connection specifies how information flows between two cells. The compiler converts this graph into an execution plan that can be evaluated using PyTorch tensors.

The initial implementation supports three cell types:

- **FFNN:** A feedforward cell with no state carried between timesteps.
- **RNN:** A recurrent cell that retains a hidden state between timesteps.
- **LSTM:** A recurrent cell that retains both a hidden state and a cell state between timesteps.

The enabled feedforward connections must form a directed acyclic graph (DAG). This allows the compiler to determine a topological execution order. Connections may skip intermediate layers: a cell can receive inputs from any earlier cell in the execution order, not only from the immediately preceding layer.

Let P_j denote the set of predecessor cells connected to cell j. Let $y_k^{(t)}$ be the output of predecessor k at timestep t.

5.1 Feedforward Cell

A feedforward cell computes its output from the current outputs of its predecessors:

$$y_j^{(t)} = f_j \left(\sum_{k \in P_j} w_{kj} y_k^{(t)} + b_j \right),$$

where w_{kj} is the connection weight, b_j is the cell bias, and f_j is the selected activation function.

A feedforward cell does not retain state between timesteps. Its output depends only on the current timestep's inputs and the outputs of its predecessors.

5.2 Recurrent Cell

A recurrent cell receives current-timestep contributions from its predecessors and also uses its own previous hidden state:

$$h_j^{(t)} = f_j \left(\sum_{k \in P_j} w_{kj} y_k^{(t)} + u_j h_j^{(t-1)} + b_j \right).$$

Here, $h_j^{(t-1)}$ is the cell's hidden state from the previous timestep, and u_j is its recurrent weight. The current output is

$$y_j^{(t)} = h_j^{(t)}.$$

The hidden state is initialized when an episode begins and retained while the episode runs. This allows the cell to incorporate information from earlier observations even when that information is not present in the current input.

5.3 Long Short-Term Memory (LSTM) Cell

An LSTM cell maintains two state variables: a hidden state h_t and a cell state c_t. Its input, forget, output, and candidate gates each have separate incoming connection weights, recurrent coefficients, and biases.

For an LSTM cell j, the gate pre-activations are

$$\begin{aligned} a_i^{(t)} &= \sum_{k \in P_j} w_{kj}^{(i)} y_k^{(t)} + u_i h^{(t-1)} + b_i, \\ a_f^{(t)} &= \sum_{k \in P_j} w_{kj}^{(f)} y_k^{(t)} + u_f h^{(t-1)} + b_f, \\ a_o^{(t)} &= \sum_{k \in P_j} w_{kj}^{(o)} y_k^{(t)} + u_o h^{(t-1)} + b_o, \\ a_g^{(t)} &= \sum_{k \in P_j} w_{kj}^{(g)} y_k^{(t)} + u_g h^{(t-1)} + b_g. \end{aligned}$$

The gate activations are

$$\begin{aligned} i_t &= \sigma(a_i^{(t)}), \\ f_t &= \sigma(a_f^{(t)}), \\ o_t &= \sigma(a_o^{(t)}), \\ g_t &= \tanh(a_g^{(t)}). \end{aligned}$$

The cell and hidden states are updated as

$$\begin{aligned} c_t &= f_t c_{t-1} + i_t g_t, \\ h_t &= o_t \tanh(c_t). \end{aligned}$$

The LSTM cell's output is $y_j^{(t)} = h_t$. The cell state provides a separate memory pathway, while the gates control how information is retained, added, and exposed through the hidden state.

5.4 Batched Execution

The simulation can evaluate multiple agents together. If a batch contains B agents, their observations and compatible cell states can be represented using tensors with a leading batch dimension. For example, a batch of B agents with d input features has an observation tensor of shape

$$X^{(t)} \in \mathbb{R}^{B \times d}.$$

The execution engine applies the appropriate cell operations to the agents in the batch. When agents have compatible network structures, their computations can be grouped into tensor operations. Different topologies or cell types may require separate execution groups or other handling by the execution engine.

5.5 Model State and Episode Reset

A compiled model contains the network parameters and execution plan, while its recurrent state is maintained separately as runtime data. At the beginning of each episode, recurrent states are reset:

$$h^{(0)} = 0, \qquad c^{(0)} = 0,$$

Skip connections are represented as ordinary feedforward connections. They do not require a special node or cell type. The compiler must aggregate contributions from all connected predecessors rather than assuming that every cell receives data only from the immediately preceding layer.

6 Simulation and Episode Evaluation

The base `Simulation` class provides a generic episode loop. Environment-specific subclasses implement the operations that depend on the simulation itself.

The generic loop performs the following steps:

1. Reset the environment and the recurrent states of the models.
2. Obtain observations for all active agents.
3. Evaluate each model to produce an action.
4. Apply the actions to the simulation.
5. Update fitness and other metrics.
6. Optionally record simulation data.
7. Repeat until all agents have terminated or the time limit is reached.
8. Finalize fitness, metrics, and optional trajectory data.

The simulation interface should provide the following environment operations:

`reset(seed=None, initial_states=None)`
Initialize the environment, optionally using a seed or supplied initial states.

`get_observations()`
Return the observations supplied to the agents' models.

`step(actions, active_mask)`
Advance the environment using the actions of agents that are still active.

`is_done()`
Determine which agents have terminated.

`get_record()`
Return the simulation-defined information to be stored by the trajectory recorder.

`evaluate()`
Return final fitness and metrics for each agent.

Agents may terminate at different timesteps. The simulation therefore maintains an active-agent mask and prevents terminated agents from continuing to affect the environment or their recurrent states.

A genome may be evaluated over multiple episodes, such as different initial conditions or random seeds. NEAT aggregates the resulting fitness values to determine the genome's fitness for that generation. Whenever possible, genomes should be evaluated using equivalent initial conditions so that fitness comparisons are fair.

7 Trajectory Recording and Replay

Three types of data should remain conceptually separate:

1. **Environment state:** The complete internal state maintained by the simulation.
2. **Observation:** The subset of information provided to the neural network at a given timestep.
3. **Trajectory record:** The information selected for storage and later analysis or visualization.

The simulation defines the contents of a record through `get_record()`. The default `set_trajectory_method()` must be overloaded by a child class to set more custom trajectory tracking logic.

The recorder should support the following policies:

- `last`: retain only the final record.
- `all`: retain every timestep.
- `interval`: retain every N-th timestep.
- `custom`: use a user-provided callback or event condition.

Tracking is separate from trajectory recording. It is an opt-in feature for selected agents that stores the initial simulation state and the actions taken at each timestep. Optional observations or additional state information may also be stored.

Replay uses the saved action sequence to advance the simulation from its recorded initial state. This avoids requiring the original neural network to be executed again.

8 Topology Visualization

The topology visualization module plots the given genome as a directed graph. It should distinguish input, hidden, and output nodes, and visually distinguish FFNN, RNN, and LSTM cells.

Connections should indicate their direction. Optional annotations include connection weights, innovation numbers, and a disabled connection should not be vizualized as part of the topology.

9 API Reference

This section defines the planned public interfaces with expected input and output data types.

9.1 NEAT Package

9.1.1 `neat/genome.py`

`Genome` Stores the node and connection genes describing one candidate network.

Signature:

```
Genome(...)
```

Arguments:

- `GenomeConfig`: instance of the GenomeConfig dataclass.

Returns:

- `Genome`: instance of the Genome class.

`Genome.mutate()` Applies mutations to the genome.

Signature:

```
mutate(...)
```

Arguments:

- `Self`: instance of the Genome class.

Returns:

- `Genome`: mutated instance of the Genome class.

`Genome.crossover()` Combines genes from two parent genomes to produce an offspring genome.

Signature:

```
crossover(...)
```

Arguments:

- `Genome`: instance of the Genome class.
- `Genome`: instance of the Genome class.

Returns:

- `Genome`: The offspring genome.

`Genome.compatibility_distance()` Calculates the compatibility distance between two genomes.

Signature:

```
compatibility_distance(...)
```

Arguments:

- `Self`: instance of the Genome class.
- `Genome`: instance of the Genome class.

Returns:

- `TODO`: Compatibility distance.

`Genome.validate()` Checks whether the genome satisfies the required structural constraints.

Signature:

```
validate(...)
```

Arguments:

- `Self`: instance of the Genome class.

Returns:

- `Bool`: model constraint satisfaction.

`Genome.serialize()` Converts the genome into a representation suitable for saving.

Signature:

```
serialize(...)
```

Arguments:

- `Self`: instance of the Genome class.

Returns:

- `None`: Saves genome representation to a file.

`GenomeConfig` Stores settings related to genome initialization and mutation.

Constructor signature:

```
GenomeConfig(...)
```

Arguments:

- `Str|Path`: Configuration parameter json file.

Returns:

- `GenomeConfig`: A genome configuration instance.

9.1.2 `neat/genes.py`

`NodeGene` Represents an individual node in a genome.

Constructor signature:

```
NodeGene(...)
```

Arguments:

- `int`: Node identifier.
- `str`: Node role.
- `str`: Cell type.
- `function`: Activation function.
- `TODO`: Node-specific parameters.

Returns:

- `NodeGene`: A node gene instance.

`ConnectionGene` Represents a directed connection between two nodes in a genome.

Constructor signature:

```
ConnectionGene(...)
```

Arguments:

- `int`: Connection identifier.
- `int`: Source node identifier.
- `int`: Target node identifier.
- `tensor`: Connection weight(s).
- `bool`: Whether the connection is enabled.
- `int`: Innovation number.

Returns:

- `ConnectionGene`: A connection gene instance.

9.1.3 `neat/innovation.py`

`InnovationTracker.get_connection_innovation()` Gets the innovation number associated with a structural connection mutation.

Signature:

```
get_connection_innovation(...)
```

Arguments:

- `ConnectionGene`: Connection gene information.

Returns:

- `int`: Innovation number.

`InnovationTracker.get_node_innovation()` Gets the innovation information associated with adding a node.

Signature:

```
get_node_innovation(...)
```

Arguments:

- `NodeGene`: Node mutation identification information.

Returns:

- `int`: Node innovation information.

9.1.4 `neat/species.py`

`Species.add()` Adds a genome to the species.

Signature:

```
add(...)
```

Arguments:

- `Genome`: Genome to add.

Returns:

- `set[Genome]`: set of all genomes in the species.

`Species.remove()` Removes a genome from the species.

Signature:

```
remove(...)
```

Arguments:

- `Genome`: Genome to remove.

Returns:

- `set[Genome]`: set of all Genomes in the species.

9.1.5 `neat/neat.py`

`NEAT.initialize_population()` Creates the initial population of genomes.

Signature:

```
initialize_population(...)
```

Arguments:

- `int`: seed.

Returns:

- `set[Genome]`: Initial population.

`NEAT.evaluate_population()` Evaluates the population using the selected simulation.

Signature:

```
evaluate_population(...)
```

Arguments:

- `set[Genome]`: Population to evaluate.

Returns:

- `list[floats]`: Evaluation results or fitness values.

`NEAT.speciate()` Assigns genomes to species based on compatibility distance.

Signature:

```
speciate(...)
```

Arguments:

- `set[Genome]`: Population to assign to species.

Returns:

- `list[set[Genome]]`: Species assignments or updated species collection.

`NEAT.reproduce()` Produces a new population through selection and reproduction.

Signature:

```
reproduce(...)
```

Arguments:

- `int`: seed.
- `set[Genome]`: population to reproduce with.

Returns:

- `set[Genome]`: New population.

`NEAT.evolve()` Runs the evolutionary process over multiple generations.

Signature:

```
evolve(...)
```

Arguments:

- `int`: seed.
- `int`: max number of generations to run.
- `patience`: max number of generations to run without improvement on fitness score.

Returns:

- `None`: results are saved as files.

`NEAT.evolve()` Runs the evolutionary process over multiple generations.

Signature:

```
evolve(...)
```

Arguments:

- `int`: seed.
- `int`: max number of generations to run.
- `patience`: max number of generations to run without improvement on fitness score.

Returns:

- `None`: results are saved as files.

9.2 Models Package

9.2.1 `models/model.py`

`Model.forward()` Evaluates the network for the supplied observations.

Signature:

```
forward(...)
```

Arguments:

- `Tensor`: the simulation observations for a single model at a given timestep.

Returns:

- `Tensor`: Model's selected action.

`Model.reset_state()` Resets the recurrent state of the model.

Signature:

```
reset_state(...)
```

Arguments:

- `None`: None

Returns:

- `None`: None

9.2.2 `models/compiler.py`

`compile_genome()` Compiles a genome into an executable model.

Signature:

```
compile_genome(...)
```

Arguments:

- `genome`: Genome to compile.

Returns:

- `Model`: instance of the Model class.

`validate_topology()` Checks whether a genome's topology can be compiled.

Signature:

```
validate_topology(...)
```

Arguments:

- `Genome`: Genome whose topology is being checked.

Returns:

- `Bool`: Topology validation result.

9.2.3 `models/cells.py`

Cell operations This module implements the tensor operations for FFNN, RNN, and LSTM cells. The specific function names and signatures are `TODO`.

9.3 Simulations Package

9.3.1 `simulations/simulation.py`

`Simulation.reset()` Initializes or resets the simulation.

Signature:

```
reset(...)
```

Arguments:

- `int`: seed.

Returns:

- `None`: None.

`Simulation.get_observations()` Returns the observations supplied to the agents' models.

Signature:

```
get_observations(...)
```

Arguments:

- `Tensor`: state of the current simulation for a given model.

Returns:

- `Tensor`: Observations for the given model's simulation.

`Simulation.step()` Advances the simulation using the agents' actions.

Signature:

```
step(...)
```

Arguments:

- `Tensor`: Actions produced by the agents' models.

Returns:

- `None`: updates the given's model simulation.

`Simulation.is_done()` Determines whether agents or the simulation have terminated.

Signature:

```
is_done(...)
```

Arguments:

- `int`: target Model instance's id in the simulation's set of models.

Returns:

- `Bool`: Termination status.

`Simulation.evaluate()` Calculates the fitness and any additional metrics for the agents.

Signature:

```
evaluate(...)
```

Arguments:

- `Tensor`: A given genome's simulation trajectory vector.

Returns:

- `Float`: Fitness score.

`Simulation.run_sim()` Runs a simulation episode using the supplied models.

Signature:

```
run_sim(...)
```

Arguments:

- `set[Genome]`: Models or agents to evaluate.
- `TODO`: Episode configuration.

Returns:

- `list[float]`: Episode evaluation results.

`Simulation.set_trajectory_method()` Sets the policy used to record simulation trajectory data.

Signature:

```
set_trajectory_method(...)
```

Arguments:

- `str|function`: Trajectory recording policy (last, all, step, custom).

Returns:

- `None`: None.

`Simulation.get_trajectory()` Returns the trajectory data recorded by the simulation.

Signature:

```
get_trajectory(...)
```

Arguments:

- `int`: The target Model's id.

Returns:

- `TODO`: Recorded trajectory.

9.4 Visualization Package

9.4.1 `visualization/topology.py`

`plot_genome()` Plots a genome as a directed graph.

Signature:

```
plot_genome(...)
```

Arguments:

- `genome`: Genome to visualize.
- `TODO`: Optional visualization settings.

Returns:

- `TODO`: Plot or visualization object.

9.4.2 `visualization/replay.py`

Replay function(s) Replays a recorded simulation trajectory.

Signature:

```
TODO
```

Arguments:

- `Simulation`: Simulation instance.
- `Tensor`: Recorded trajectory or replay data.
- `TODO`: Optional replay settings.

Returns:

- `TODO`: TODO.

References

[1] [Online]. Available: https://dl.acm.org/doi/10.1145/2908812.2908941

[2] [Online]. Available: https://nn.cs.utexas.edu/?stanley:ec02